AR Sudoku: App and Website Design

Rescoped version, November 12

Muhammad Ali Hassan

1 Goal

One link that opens on any device (a Mac, an iPhone, an Android phone) with nothing to install. You point the camera at a Sudoku. The website finds the puzzle, solves it, and writes the missing digits on the paper. It also works on a puzzle you already started: it checks your handwritten digits and fills in the rest.

New: if you have no Sudoku on paper, the app draws one in AR on any flat surface: a table, a wall, a whiteboard or a sheet of paper. You drag the 4 corners of the board to where you want it, and the board stays there when the phone moves. You play by tapping cells, or by writing with a pen when the board is on paper.

Two rules for the whole project:

- **Free hosting.** A static site on GitHub Pages: `alihassanmirza1.github.io/AR-Sudoku-Solver-with-Computer-Vision`
- **All computing on the device that opens the page.** No server. The video never leaves the device, and the site works offline after the first visit.
- **Leonardo:** vision and solver (`sudoku_cv/`).
- **Me:** the website. Camera, screens, drawing digits on the paper, the virtual board, hosting.

Classical computer vision only, no neural networks.

What changed from the first version

Topic	Change
Hosting	GitHub Pages was the plan; now it is a requirement. One URL for Mac, iPhone and Android.
Computing	Must stay on the device. The "Hugging Face server" fallback is removed; every fallback now runs on the device.
New mode	Virtual board placed with draggable corners on any flat surface (section 2).
Plan	Re-timed from today. Leonardo's real code is already pushed, so I skip the stub and build on his real code.

2 Two modes

A. Paper puzzle (same as before). Point at a printed Sudoku. Live shows all missing digits one after another; Step shows one per tap, as a hint. Check freezes the picture so the user can fix a

misread digit. Continue colours the user's handwritten digits green (right) or red (wrong). **New:** if the app cannot find the grid, or finds it wrong, the user can set its 4 corners by hand with the same handles as in mode B.

B. Virtual board (new).

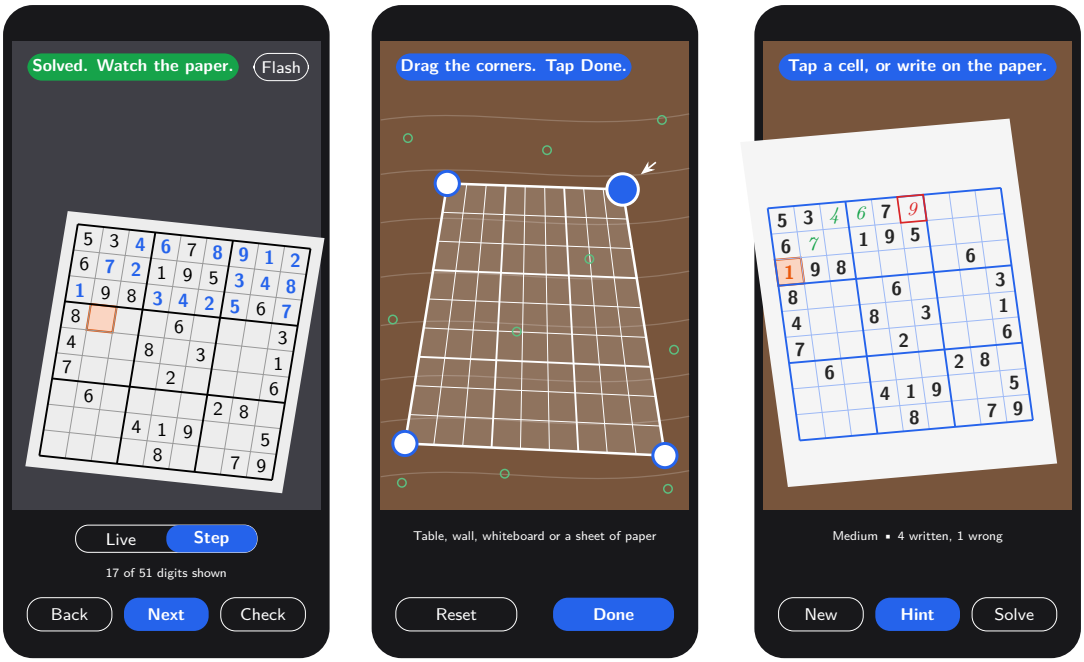
1. Tap *Draw a board* and pick a puzzle: a new one (easy, medium or hard), or type one in by tapping cells (for example, a newspaper puzzle you don't want to write on).
2. **Place it.** Point the camera at any flat surface. An empty board appears with a round handle on each corner. Drag each corner to where you want it: on a table, a wall, a whiteboard or a sheet of paper. Drag the middle to move the whole board. The 4 corners can form any 4-sided shape, so the user makes the board look square on the surface as seen from the camera, and the homography does the rest. If the app sees a sheet of paper, the corners first snap to the sheet. Tap *Done*.
3. **It stays there.** The app keeps the board on the surface while the phone moves (see below).
4. **Play.** Tap a cell on the screen and pick a digit. This works on any surface. On paper or a whiteboard the user can also write with a pen or marker; when the board is in view and still, the app reads the new digits. Digits are coloured green or red, as in Continue mode. *Hint* shows one digit in orange, *Solve* shows all of them.
5. If the board slips or is lost, tap *Adjust* and drag the corners again.
6. Optional *Trace* (on paper): the user draws over the board lines with a pen and a ruler. The paper now has a real board, and mode A takes over.

How the board stays on the surface. When the user taps Done, the app keeps that frame as the *reference* and the 4 corners in it. Then, all with classical CV:

- **Every frame:** Lucas–Kanade optical flow follows about 100 strong points of the surface (corners found with `goodFeaturesToTrack`) inside and around the board. A RANSAC homography from these points moves the 4 board corners. Points hidden by a hand are thrown out by RANSAC.
- **Every few frames, or when many points are lost:** ORB features of the new frame are matched to the reference frame, and a RANSAC homography from the reference gives the corners again. This removes the slow drift of optical flow and finds the board again after it left the view.
- **On a sheet of paper:** if the corners snapped to a sheet, the sheet's 4 corners are found again in every frame with the contour method from Leonardo's `detect.py`. This works even though a blank sheet has no texture inside.
- **Plain surface** (a white wall): there are too few points to follow. The board stays at the same place on the screen and the message says to hold the phone still; it works well with the phone on a stand. Points around the board count too, so a poster or a picture frame next to the board is enough.

Phone AR frameworks (WebXR plane detection) do not run in iPhone Safari, so I do not use them. The methods above are all in the course (features, optical flow, RANSAC, homography) and run in Pyodide's OpenCV.

3 Screens



Paper puzzle, Step mode | Placing a virtual board | Virtual board, playing

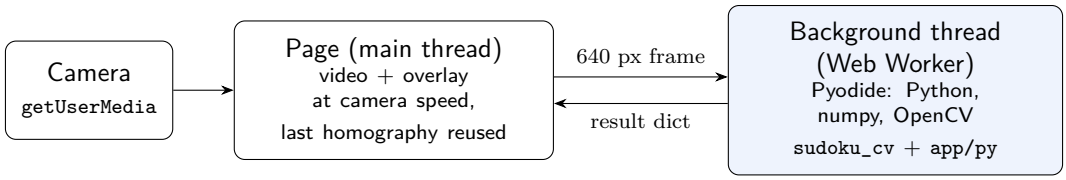
Green circles: surface points the tracker follows.

- **Home:** start the camera, draw a virtual board, try a sample puzzle, or upload a photo.
- **Colours:** printed digits black, the app's digits blue, the next one orange, the user's digits green when right and red when wrong. On the virtual board the lines are blue (or white while placing) so it is clear they are not really on the surface.
- **Install:** "Add to Home Screen" opens the site full screen, like an app.

What the code reports	Message on screen
no puzzle found	Point the camera at a Sudoku
puzzle found, reading	Hold still…
can't be solved	Some digits look wrong. Tap Check.
solved	Solved. Watch the paper.
placing the board (virtual)	Drag the corners. Tap Done.
sheet of paper seen while placing	Corners snapped to the sheet. Tap Done.
board placed	Tap a cell, or write on the paper
board partly hidden (hand)	Board kept; checking when your hand moves away
too few surface points	Plain surface: hold the phone still
board lost	Point back at the board, or tap Adjust

4 How it works

1. The camera gives video frames. The page always shows the live video.
2. When the background thread is free, the page takes the newest frame, shrinks it to 640 px and sends it (RGBA pixels, no copy). Only one frame is in flight; the others are skipped, so



the video never freezes.

3. In the background thread, Leonardo's Python runs inside the browser with Pyodide (WebAssembly). It returns the result dict: status, 4 corners, homography, digits, solution, reveal order, wrong digits.
4. The page writes the digits on a flat 450×450 square and bends it onto the paper with the homography (a CSS `matrix3d` transform). Between results it keeps the last homography, so drawing stays smooth even if Python is slower than the camera.
5. **Virtual board:** while placing, the handles are plain page elements and need no Python. After Done, the worker runs the surface tracker (section 2) instead of the Sudoku tracker and returns the board's 4 corners and homography, so drawing is the same as in step 4. A tap on the screen goes through the inverse homography to find the cell. To read pen digits, the same homography flattens the board back to 450×450, so Leonardo's cell reader works unchanged.

Hosting and deploy

- GitHub Pages gives HTTPS, which browsers require before they allow the camera.
- A GitHub Action runs on every push to `main`: it zips `sudoku_cv/` (code and `.npz` data) next to `app/` and publishes the site. Leonardo's changes go live without either of us copying files.
- The Pyodide files (Python, numpy, OpenCV) are served from our own site with a fixed version, not from a CDN, so a service worker can cache everything for offline use. It is a few tens of MB, within GitHub Pages limits (100 MB per file, 1 GB per site). I measure the exact size in week 1.
- Free GitHub Pages needs a public repository (or GitHub Education for a private one).

Devices

Device	Notes
Mac (Safari, Chrome)	The built-in camera faces the user: hold the paper up to it, or use an iPhone as a webcam (Continuity Camera). The virtual board can go on anything the camera sees, such as the wall behind. Photo upload always works.
iPhone (Safari)	Back camera via `facingMode: "environment"`, `playsinline` video, camera starts only after a tap. Memory is the main risk for Pyodide: tested first.
Android (Chrome)	No special cases expected.

5 Speed: what I measured on Leonardo's code

I ran his pushed code natively on my Mac (Python, OpenCV) on a synthetic 640×480 frame of a tilted printed puzzle:

- `process_image` (single photo, read + solve): **112 ms**, all 30 printed digits read correctly, solved.
- `SudokuTracker.process_frame` (live, per frame): **about 70–90 ms**. About 97% of that is `refine_corners`, which runs on every frame and calls `warpPerspective` about 440 times per frame.

Python in the browser is usually several times slower than native, so as it is now the live mode would show only a few updates per second on a phone. The plan, all on the device:

1. Skip `refine_corners` when the new corners are close to the last ones, or refine only on the frames where digits are read (asked of Leonardo, section 6). Without it the per-frame path should drop to a few ms natively.
2. The overlay already runs at camera speed with the last homography (step 4 above), so a slower Python only makes the overlay lag, never freeze.
3. If still too slow: 480 px frames, and follow the 4 corners between Sudoku tracker results with the same optical flow as the virtual board.

Surface tracking for the virtual board (same Mac, 640×480 textured test frame): ORB with 500 features + matching + RANSAC homography takes **7 ms**; optical flow on 100 points + RANSAC takes **under 1 ms**. Even several times slower in the browser, this fits in every frame on a phone.

6 What I need from Leonardo

His push already gives most of what the first version asked for: `process_image`, `SudokuTracker` (`process_frame`, `set_givens`, `reset`), `solve(givens, user_digits)`, a confidence per cell, the `wrong` list, handwriting read with HOG and k-nearest neighbours, and the debug images. New requests:

1. **Speed:** the `refine_corners` change above.
2. **Corners set by hand (mode A):** `tracker.set_corners(corners)` and a `corners` argument for `process_image`: skip grid detection, read the digits inside the 4 corners the user placed.
3. **Reading a virtual board:** `read_handwritten(frame, homography)`: flatten the board, and read every cell that has ink as handwriting (there are no printed digits on the paper, they are only on screen). Returns `user_digits` and `confidence`. I call it only when the board has been still for a few frames.
4. **Keep OpenCV 4.x:** Pyodide ships OpenCV 4, and `requirements.txt` already says `opencv-python<5`. I will check in week 1 that everything his code calls (including `cv2.imencode` for the debug images) exists in Pyodide's build.

If he is short on time, I can write items 2 and 3 in `app/py/` on top of his `detect.py` and `digits.py` functions, and he reviews them.

What I write myself:

- **Surface tracker** in `app/py/track.py`: snapping to a sheet of paper (on top of his `threshold` and `order_corners`), optical flow, ORB matching, RANSAC homography, and the "lost / plain surface" states. It belongs to the virtual board, so it is mine; it also gives me a vision part of my own for the report.
- **Puzzle generator** in `app/py/generate.py`, using his solver. Fill an empty grid with random backtracking, then remove digits one at a time while the puzzle still has exactly one solution.

Difficulty is the number of givens (easy 40, medium 32, hard 26). Typed-in puzzles go through `solve` to check that they have a solution.

7 Repository

```
app/            my code: index.html, app.js, worker.js, overlay.js,
                virtual.js, sw.js (offline), manifest.json (install)
app/py/         bridge.py (calls sudoku_cv from the worker),
                track.py (surface tracker), generate.py
sudoku_cv/      Leonardo's package (I do not edit it)
samples/        test photos and videos (not committed)
.github/workflows/pages.yml   build and publish to GitHub Pages
```

8 Steps to finish

	Dates	What I do
1	Nov 12 – 15	Site and GitHub Action live on Pages. Worker loads Pyodide + OpenCV and runs Leonardo's real `process_image` on a sample photo. Time it on the Mac, an iPhone and an Android phone. **Nov 15:** decide which speed fixes from section 5 we need.
2	Nov 16 – 20	Camera on all devices, frame loop, overlay with `matrix3d`, messages, Live and Step.
3	Nov 21 – 25	Virtual board: corner handles, snapping to a sheet, surface tracker (optical flow, ORB, RANSAC), tap input, generator, typed-in puzzles, reading pen digits, Hint and Solve.
4	Nov 26 – 29	Check screen, Continue colours, photo upload, offline + install. Test on the 3 devices, 5 classmates try it. (Nov 26 is Thanksgiving.)
5	Nov 30	Code freeze.
6	Dec 1 – 11	Demo on Dec 1 or 3 with a backup video. Report due Dec 11.

If we run late, cut in this order: Trace, the "How it works" debug view, typed-in puzzles, reading pen digits on the virtual board (tapping stays). Not cut: the paper puzzle mode, and the virtual board with corner handles, tracking, generated puzzles and tap input.

9 Testing

On a Mac, an iPhone and an Android phone:

- **Speed:** overlay updates per second (target: 15 on a laptop, 10 on phones) and Python results per second.
- **Load time:** first visit on Wi-Fi (target under 15 s, Pyodide is large), under 2 s after. Works with no network after the first visit.
- **Time to solve:** from "Start camera" to solved, on 10 printed puzzles.
- **Steadiness:** how much the digits shake while the paper is still, in both modes.
- **Continue mode:** on 10 half-filled puzzles, how many wrong digits are caught.

- **Virtual board:** placed on 5 surfaces (wood table, plain wall, wall with a poster, whiteboard, sheet of paper). For each: time to place it, how far the board drifts (in pixels) after 30 s of moving the phone, and whether it is found again after leaving the view. Pen digits read right on 3 paper boards filled by 3 different people; board kept while a hand covers one corner.
- **Ease of use:** 5 classmates try both modes with no help.

10 Risks

- **Too slow on phones:** the fixes in section 5, all on the device. No server.
- **iPhone memory or camera issues:** tested in week 1; photo upload always works.
- **Mac camera faces the user:** hold the paper up, use the iPhone as a webcam, or upload a photo.
- **Plain wall or table, nothing to follow:** the board stays fixed on the screen and the user holds the phone still or uses a stand; a sheet of paper or a poster nearby fixes it.
- **Board drifts or is lost after fast motion:** ORB against the reference frame finds it again; if not, Adjust.
- **Hand covers the board while writing:** RANSAC ignores the hidden points; pen digits are read only when the board is back in full view and still.
- **Handwriting read wrong:** the user taps the cell to fix it.
- **Demo fails in class:** recorded backup video.